EXPERIMENT NO:- 1

```
#include <stdio.h>
#include <conio.h>
void main()
{
   int a[50], n, i, j, key;
   clrscr();
   printf("Enter the number of elements: ");
   scanf("%d", &n);
   printf("Enter %d elements:\n", n);
   for(i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }
   /* Insertion Sort */
   for(i = 1; i < n; i++)
   {
      key = a[i];
      j = i - 1;
      while(j >= 0 && a[j] > key)
      {
         a[j + 1] = a[j];
         j--;
      }
      a[j + 1] = key;
   }
   printf("\nSorted array in ascending order:\n");
   for(i = 0; i < n; i++)
   {
      printf("%d ", a[i]);
   }
   getch();
}
```

Output:-

```
Enter the number of elements: 6
Enter 6 elements:
9
3
6
1
7
2

Sorted array in ascending order:
1 2 3 6 7 9
```

EXPERIMENT NO:- 2

```
#include <stdio.h>
#include <conio.h>
void main()
{
   int a[50], n, i, key;
   int low, high, mid, found = 0;
   clrscr();
   printf("Enter the number of elements: ");
   scanf("%d", &n);
   printf("Enter %d elements in sorted order:\n", n);
   for(i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }
   printf("Enter the element to search: ");
   scanf("%d", &key);
   low = 0;
   high = n - 1;
   while(low <= high)
   {
      mid = (low + high) / 2;
      if(a[mid] == key)
      {
         found = 1;
         break;
      }
      else if(key < a[mid])
      {
         high = mid - 1;
      }
      else
      {
         low = mid + 1;
      }
```

```
    }

    if(found)
        printf("\nElement found at position %d", mid + 1);
    else
        printf("\nElement not found.");
    getch();
}
```

Output:-

```
Enter the number of elements: 6
Enter 6 elements in sorted order:
10
20
30
50
60
80
Enter the element to search: 50

Element found at position 4
```

EXPERIMENT NO:- 3

```
#include <stdio.h>
#include <conio.h>

void merge(int a[ ], int low, int mid, int high)
{
   int temp[50];
   int i, j, k;
   i = low;
   j = mid + 1;
   k = low;
   while(i <= mid && j <= high)
   {
      if(a[i] < a[j])
         temp[k++] = a[i++];
      else
         temp[k++] = a[j++];
   }
   while(i <= mid)
      temp[k++] = a[i++];
   while(j <= high)
      temp[k++] = a[j++];
   for(i = low; i <= high; i++)
      a[i] = temp[i];
}
void mergeSort(int a[], int low, int high)
{
   int mid;

   if(low < high)
   {
      mid = (low + high) / 2;
      mergeSort(a, low, mid);
      mergeSort(a, mid + 1, high);
      merge(a, low, mid, high);
```

```
    }
}
void main()
{
    int a[50], n, i;
    clrscr();
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    mergeSort(a, 0, n - 1);
    printf("\nSorted array using Merge Sort:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    getch();
}
```

Output:-

```
Enter the number of elements: 6
Enter 6 elements:
13
24
56
76
45
56

Sorted array using Merge Sort:
13 24 45 56 56 76
```

EXPERIMENT NO:- 4

```
#include <stdio.h>
#include <conio.h>
void quicksort(int a[], int low, int high)
{
   int i, j, pivot, temp;
   if(low < high)
   {
      pivot = a[low];
      i = low + 1;
      j = high;
      while(i <= j)
      {
         while(i <= high && a[i] <= pivot)
            i++;
         while(a[j] > pivot)
            j--;
         if(i < j)
         {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
         }
      }

      a[low] = a[j];
      a[j] = pivot;
      quicksort(a, low, j - 1);
      quicksort(a, j + 1, high);
   }
}
void main()
{
   int a[50], n, i;
```

```
    clrscr();
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    quicksort(a, 0, n - 1);
    printf("\nSorted array using Quick Sort:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    getch();
}
```

Output:-

```
Enter the number of elements: 6
Enter 6 elements:
23
443
12
4
56
78

Sorted array using Quick Sort:
4 12 23 56 78 443
```

EXPERIMENT NO:- 6

```
#include <stdio.h>
#include <conio.h>
#define MAX 20
int parent[MAX];
int find(int i)
{
   while(parent[i] != i)
      i = parent[i];
   return i;
}
void uni(int i, int j)
{
   parent[i] = j;
}
void main()
{
   int cost[MAX][MAX];
   int i, j, n;
   int min, a, b, u, v;
   int ne = 1;
   int mincost = 0;
   clrscr();
   printf("Enter the number of vertices: ");
   scanf("%d", &n);
   printf("Enter the cost adjacency matrix:\n");
   for(i = 1; i <= n; i++)
   {
      parent[i] = i;
      for(j = 1; j <= n; j++)
      {
         scanf("%d", &cost[i][j]);

         if(cost[i][j] == 0)
```

```
            cost[i][j] = 999;
        }
    }
    printf("\nEdges in the Minimum Spanning Tree:\n");
    while(ne < n)
    {
        min = 999;
        for(i = 1; i <= n; i++)
        {
            for(j = 1; j <= n; j++)
            {
                if(cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }
            }
        }
        u = find(u);
        v = find(v);
        if(u != v)
        {
            printf("%d Edge (%d,%d) = %d\n", ne++, a, b, min);
            mincost += min;
            uni(u, v);
        }
        cost[a][b] = cost[b][a] = 999;
    }
    printf("\nMinimum Cost = %d", mincost);
    getch();
}
```

Output:-

```
Enter the number of vertices: 4
Enter the cost adjacency matrix:
0 12 5 10
9 10 0 0
6 0 5 7
10 5 16 15

Edges in the Minimum Spanning Tree:
1 Edge (1,3) = 5
2 Edge (4,2) = 5
3 Edge (3,4) = 7

Minimum Cost = 17
```

EXPERIMENT NO:- 5

```
#include <stdio.h>
#include <conio.h>
struct Item
{
   int weight;
   int profit;
   float ratio;
};
void main()
{
   struct Item item[10], temp;
   int i, j, n, capacity;
   float totalProfit = 0.0;
   clrscr();
   printf("Enter number of items: ");
   scanf("%d", &n);
   for(i = 0; i < n; i++)
   {
      printf("\nEnter Profit and Weight of item %d: ", i + 1);
      scanf("%d%d", &item[i].profit, &item[i].weight);
      item[i].ratio = (float)item[i].profit / item[i].weight;
   }
   printf("\nEnter Knapsack Capacity: ");
   scanf("%d", &capacity);
   /* Sort items by Profit/Weight ratio in descending order */
   for(i = 0; i < n - 1; i++)
   {
      for(j = i + 1; j < n; j++)
      {
         if(item[i].ratio < item[j].ratio)
         {
            temp = item[i];
            item[i] = item[j];
            item[j] = temp;
```

```
            }
        }
    }
    printf("\nSelected Items:\n");
    for(i = 0; i < n; i++)
    {
        if(capacity >= item[i].weight)
        {
            printf("Take Item %d completely\n", i + 1);
            capacity = capacity - item[i].weight;
            totalProfit = totalProfit + item[i].profit;
        }
        else
        {
            printf("Take %.2f fraction of Item %d\n",
                  (float)capacity / item[i].weight, i + 1);

            totalProfit = totalProfit +
                        item[i].profit *
                        ((float)capacity / item[i].weight);
            break;
        }
    }
    printf("\nMaximum Profit = %.2f", totalProfit);
    getch();
}
```

Output:-

```
Enter number of items: 3

Enter Profit and Weight of item 1: 20 30

Enter Profit and Weight of item 2: 10 100

Enter Profit and Weight of item 3: 50 90

Enter Knapsack Capacity: 60

Selected Items:
Take Item 1 completely
Take 0.33 fraction of Item 2

Maximum Profit = 36.67
```

EXPERIMENT NO:- 7

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
int x[4];
int count = 0;
/* Check whether queen can be placed */
int place(int k, int i)
{
    int j;
    for (j = 0; j < k; j++)
    {
        if (x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}
/* Display chess board */
void display()
{
    int i, j;
    printf("\nSolution %d:\n", count);
    for (i = 0; i < 4; i++)
    {
        printf("+---+---+---+---+\n");
        for (j = 0; j < 4; j++)
        {
            if (x[i] == j)
                printf("| Q ");
            else
                printf("|   ");
        }
        printf("|\n");
    }
    printf("+---+---+---+---+\n");
}
```

```
/* 4-Queen Backtracking */
void nqueen(int k)
{
   int i;
   for (i = 0; i < 4; i++)
   {
      if (place(k, i))
      {
         x[k] = i;
         if (k == 3)
         {
            count++;
            display();
         }
         else
         {
            nqueen(k + 1);
         }
      }
   }
}
void main()
{
   clrscr();
   printf("4-QUEEN PROBLEM");
   printf("\n================");
   nqueen(0);
   printf("\nTotal Solutions = %d", count);
   getch();
}
```

Output:-

```
4-QUEEN PROBLEM
===============
Solution 1:
+---+---+---+---+
|   | Q |   |   |
+---+---+---+---+
|   |   |   | Q |
+---+---+---+---+
| Q |   |   |   |
+---+---+---+---+
|   |   | Q |   |
+---+---+---+---+

Solution 2:
+---+---+---+---+
|   |   | Q |   |
+---+---+---+---+
| Q |   |   |   |
+---+---+---+---+
|   |   |   | Q |
+---+---+---+---+
|   | Q |   |   |
+---+---+---+---+

Total Solutions = 2
```

EXPERIMENT NO:- 8

```
#include <stdio.h>
#include <conio.h>
#define MAX 10
int graph[MAX][MAX];
int color[MAX];
int n, m;
/* Function to check whether a color can be assigned */
int isSafe(int v, int c)
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (graph[v][i] == 1 && color[i] == c)
            return 0;
    }
    return 1;
}
/* Function to solve graph coloring */
int graphColoring(int v)
{
    int c;
    if (v == n)
        return 1;
    for (c = 1; c <= m; c++)
    {
        if (isSafe(v, c))
        {
            color[v] = c;
            if (graphColoring(v + 1))
                return 1;
            color[v] = 0;
        }
    }
    return 0;
}
```

```
void main()
{
    int i, j;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of colors: ");
    scanf("%d", &m);
    printf("\nEnter the adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    for (i = 0; i < n; i++)
        color[i] = 0;
    if (graphColoring(0))
    {
        printf("\nGraph can be colored using %d colors.\n", m);
        printf("\nVertex\tColor\n");
        for (i = 0; i < n; i++)
        {
            printf("%d\t%d\n", i + 1, color[i]);
        }
    }
    else
    {
        printf("\nGraph cannot be colored using %d colors.", m);
    }

    getch();
}
```

Output:-

```
Enter number of vertices: 4
Enter number of colors: 3

Enter the adjacency matrix:
0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0

Graph can be colored using 3 colors.

Vertex  Color
1       1
2       2
3       3
4       2
```

EXPERIMENT NO:- 9

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#define MAX 50
void main()
{
   char X[MAX], Y[MAX];
   int L[MAX][MAX];
   int i, j;
   int m, n;
   clrscr();
   printf("Enter first string: ");
   scanf("%s", X);
   printf("Enter second string: ");
   scanf("%s", Y);
   m = strlen(X);
   n = strlen(Y);
   /* Build LCS table */
   for (i = 0; i <= m; i++)
   {
      for (j = 0; j <= n; j++)
      {
         if (i == 0 || j == 0)
            L[i][j] = 0;
         else if (X[i - 1] == Y[j - 1])
            L[i][j] = L[i - 1][j - 1] + 1;
         else
         {
            if (L[i - 1][j] > L[i][j - 1])
               L[i][j] = L[i - 1][j];
            else
               L[i][j] = L[i][j - 1];
         }
      }
```

```
    }
    printf("\nLength of LCS = %d", L[m][n]);
    /* Print LCS */
    {
        char lcs[MAX];
        int index = L[m][n];
        lcs[index] = '\0';
        i = m;
        j = n;
        while (i > 0 && j > 0)
        {
            if (X[i - 1] == Y[j - 1])
            {
                lcs[index - 1] = X[i - 1];
                i--;
                j--;
                index--;
            }
            else if (L[i - 1][j] > L[i][j - 1])
                i--;
            else
                j--;
        }
        printf("\nLCS = %s", lcs);
    }
    getch();
}
```

Output:-

